

ML stats application

1 Fundamental Concepts

Big-O Quick Rules

- Ignore constants and lower-order terms: $3n^2 + 10n \in \Theta(n^2)$.
- Sequential code: add costs. Nested loops: multiply (often $\times n$ per loop).
- Conditionals: take the max branch (worst-case), or weighted avg (expected).
- Log base doesn't matter: $\log_a n = \Theta(\log n)$.
- Typical growth (small to large): $1 < \log n < n < n \log n < n^2 < n^3 < 2^n < n!$.

Master Theorem (Common Form)

For $T(n) = aT(n/b) + f(n)$ with $a \geq 1$, $b > 1$:

$n^{\log_b a}$ = “divide & conquer critical exponent”

- If $f(n) = O(n^{\log_b a - \epsilon})$ then $T(n) = \Theta(n^{\log_b a})$.
- If $f(n) = \Theta(n^{\log_b a} \log^k n)$ then $T(n) = \Theta(n^{\log_b a} \log^{k+1} n)$.
- If $f(n) = \Omega(n^{\log_b a + \epsilon})$ and regularity holds then $T(n) = \Theta(f(n))$.

Handy Sums

$$\sum_{i=1}^n i = \Theta(n^2), \quad \sum_{i=1}^n i^2 = \Theta(n^3), \quad \sum_{i=0}^{\lfloor \log n \rfloor} 1 = \Theta(\log n)$$
$$\sum_{i=0}^{\lfloor \log n \rfloor} 2^i = \Theta(n), \quad \sum_{i=1}^n \frac{1}{i} = \Theta(\log n)$$

Core Data Structures

Arrays / Dynamic Arrays (e.g., Python list, Java ArrayList)

- Access by index: $\Theta(1)$
- Append: amortized $\Theta(1)$, worst-case $\Theta(n)$ on resize
- Insert/Delete at position: $\Theta(n)$ (shift)
- Search (unsorted): $\Theta(n)$; (sorted): $\Theta(\log n)$ via binary search

Hash Table / Dict

- Insert/Lookup/Delete: expected $\Theta(1)$, worst-case $\Theta(n)$ (rare; adversarial/collisions)

Stack / Queue / Deque

- Push/Pop/Enqueue/Dequeue: $\Theta(1)$ (amortized for dynamic backing)

Linked List

- Insert/Delete given node pointer: $\Theta(1)$
- Search / access by index: $\Theta(n)$

Binary Heap (Priority Queue)

- Find-min/max: $\Theta(1)$
- Insert: $\Theta(\log n)$
- Extract-min/max: $\Theta(\log n)$
- Decrease-key: $\Theta(\log n)$ (implementation dependent)
- Build-heap from array: $\Theta(n)$

Balanced BST (AVL/Red-Black)

- Search/Insert/Delete: $\Theta(\log n)$
- Ordered iteration / predecessor / range queries: $\Theta(\log n + k)$ (k outputs)

Union-Find (Disjoint Set Union, DSU)

With union by rank + path compression:

amortized $\Theta(\alpha(n)) \approx$ “almost constant”

Sorting and Selection

Comparison Sorting (lower bound)

Any comparison sort: $\Omega(n \log n)$ comparisons (worst-case).

Common Sorts

- MergeSort: $\Theta(n \log n)$ time, $\Theta(n)$ extra space, stable
- QuickSort: expected $\Theta(n \log n)$, worst-case $\Theta(n^2)$, in-place-ish
- HeapSort: $\Theta(n \log n)$ time, $\Theta(1)$ extra space, not stable
- TimSort (Python/Java): worst-case $\Theta(n \log n)$, exploits runs (often near $\Theta(n)$)

Non-comparison Sorts (when keys are bounded)

- Counting Sort: $\Theta(n + k)$ time, $\Theta(n + k)$ space, keys in $\{1, \dots, k\}$
- Radix Sort: $\Theta(d(n + k))$ where d digits/passes (base-dependent)
- Bucket Sort: expected $\Theta(n)$ under uniform assumptions; worst-case $\Theta(n^2)$

Selection

- QuickSelect (kth element): expected $\Theta(n)$, worst-case $\Theta(n^2)$
- Median of Medians: worst-case $\Theta(n)$ (larger constant)

Searching

- Linear search: $\Theta(n)$
- Binary search (sorted array): $\Theta(\log n)$
- Two pointers on sorted array: typically $\Theta(n)$
- Sliding window: typically $\Theta(n)$ (each index enters/leaves window once)

String Algorithms

Let $n = |text|$, $m = |pattern|$.

- Naive substring search: worst-case $\Theta(nm)$
- KMP: $\Theta(n + m)$
- Rabin-Karp: expected $\Theta(n + m)$, worst-case $\Theta(nm)$ (hash collisions)
- Trie insert/lookup: $\Theta(m)$
- Sorting strings: $\Theta(n \log n \cdot L)$ where L avg compare length (can be optimized)

Graph Algorithms

Assume adjacency list unless stated.

Traversals

- BFS: $\Theta(V + E)$ time, $\Theta(V)$ space
- DFS: $\Theta(V + E)$ time, $\Theta(V)$ space (stack/recursion)

Shortest Paths

- Dijkstra (non-negative weights):
 - with binary heap: $\Theta((V + E) \log V)$
 - with Fibonacci heap (rare in interviews): $\Theta(E + V \log V)$
- Bellman-Ford: $\Theta(VE)$ (handles negative edges; detects negative cycles)
- Floyd-Warshall: $\Theta(V^3)$ (all-pairs; dense graphs)
- DAG shortest path (topo order): $\Theta(V + E)$

Minimum Spanning Tree

- Kruskal: $\Theta(E \log E)$ (plus DSU $\alpha(V)$)
- Prim (binary heap): $\Theta(E \log V)$

Topological Sort

- Kahn's algorithm / DFS-based: $\Theta(V + E)$

Connectivity / Components

- Connected components (undirected): BFS/DFS $\Theta(V + E)$
- Strongly connected components: Kosaraju $\Theta(V + E)$, Tarjan $\Theta(V + E)$

Dynamic Programming (DP) Patterns

Common DP Table Sizes

- 1D DP over n : typically $\Theta(n)$ time and $\Theta(n)$ space (often compressible to $\Theta(1)$)
- 2D DP over (n, m) : $\Theta(nm)$ time, $\Theta(nm)$ space (often compressible to $\Theta(\min(n, m))$)

Canonical Examples

- Knapsack (0/1): $\Theta(nW)$ where W is capacity
- Longest Common Subsequence (LCS): $\Theta(nm)$
- Edit distance: $\Theta(nm)$

Backtracking / Exponential

- Subsets: $\Theta(2^n)$
- Permutations: $\Theta(n!)$
- Backtracking often described as $\Theta(b^d)$ where b branching factor, d depth.
- Pruning helps in practice but worst-case usually stays exponential.

Amortized Analysis (Interview Favorites)

Dynamic Array Append

Even if resize costs $\Theta(n)$, total cost over n appends is $\Theta(n) \Rightarrow$ amortized $\Theta(1)$.

Monotonic Stack / Queue

Each element is pushed and popped at most once $\Rightarrow$ total $\Theta(n)$.

Space Complexity Quick Notes

- Recursion depth: DFS on a path is $\Theta(V)$ stack space.
- Adjacency list: $\Theta(V + E)$; adjacency matrix: $\Theta(V^2)$.
- Sorting: MergeSort uses $\Theta(n)$ extra; HeapSort $\Theta(1)$ extra.

Tiny Checklist for Interview Answers

- State what n is (items? nodes? edges? length?).
- Specify assumptions: adjacency list vs matrix; expected vs worst-case; bounded keys.
- Give both time and space. Mention amortized/expected if relevant.

Python Collections Cheat Sheet (Algorithmic View)

List list (Dynamic Array)

```
arr = []

arr.append(x)      # amortized  $O(1)$ 
arr.pop()          #  $O(1)$ 
arr.insert(i, x)   #  $O(n)$  (shift elements)
arr[i]             #  $O(1)$  (random access)
```

Notes:

- Backed by a dynamic array
 - Appends are amortized constant time
 - Avoid `pop(0)` ($O(n)$)
-

Deque collections.deque (Double-Ended Queue)

```
from collections import deque
q = deque()
```

```
q.append(x)      #  $O(1)$ 
q.appendleft(x)  #  $O(1)$ 
q.popleft()      #  $O(1)$ 
```

Notes:

- Implemented as a doubly-linked list of blocks
 - Ideal for BFS / sliding window
-

Set set (Hash Set)

```
s = set()

s.add(x)          # average  $O(1)$ 
x in s            # average  $O(1)$ 
s.remove(x)       # average  $O(1)$ 
```

Notes:

- Unordered, no duplicates
 - Backed by a hash table
-

Dict dict (Hash Table / Map)

```
m = {}
```

```
m[key] = value      # average O(1)
value = m[key]      # average O(1)
key in m            # average O(1)
```

Notes:

- Keys must be hashable
 - Preserves insertion order (Python 3.7+)
-

Heap heapq (Binary Min-Heap)

```
import heapq
h = []

heapq.heappush(h, x) # O(log n)
heapq.heappop(h)     # O(log n)
h[0]                 # O(1) (min element)
```

Notes:

- Min-heap only (use negative values for max-heap)
 - Common for top-K, scheduling, Dijkstra
-

Counter collections.Counter (Hash Map for Frequencies)

```
from collections import Counter
c = Counter(arr)

c[x]          # O(1)
c.most_common(k) # O(n log k)
```

Notes:

- Specialized dict for counting
 - Very common in string / frequency problems
-

DefaultDict collections.defaultdict (Hash Map)

```
from collections import defaultdict
g = defaultdict(list)

g[u].append(v)      # O(1)
```

Notes:

- Eliminates key-existence checks
 - Standard for graph adjacency lists
-

Summary Table (Mental Model)

Python Type	Algorithmic Structure
<code>list</code>	Dynamic array
<code>deque</code>	Double-ended queue
<code>set</code>	Hash set
<code>dict</code>	Hash table (map)
<code>heapq</code>	Binary heap
<code>Counter</code>	Hash map (frequency table)
<code>defaultdict</code>	Hash map with default values

Substring Search: KMP (Knuth–Morris–Pratt)

Given:

- Text of length n
- Pattern of length m

Idea: Precompute an LPS (longest proper prefix which is also a suffix) array for the pattern. On mismatch, reuse previous matches instead of restarting.

Preprocessing (LPS array):

$$\text{Time} = \Theta(m), \quad \text{Space} = \Theta(m)$$

Search:

$$\text{Time} = \Theta(n), \quad \text{Space} = \Theta(1)$$

Total Complexity:

$$\Theta(n + m)$$

Key Property: Each character in the text is processed at most once.

KMP Substring Search (With Comments)

Goal: Find the first occurrence of pattern P (length m) in text T (length n) in $\Theta(n + m)$ time.

Step 1: Build LPS Array

LPS[i] = length of the longest proper prefix of P[0..i] that is also a suffix of P[0..i]

```
LPS[0] = 0           // A single character has no proper prefix
len = 0              // Length of current candidate prefix
i = 1                // Start from second character
```

```
while i < m:
    if P[i] == P[len]:
        // We can extend the current prefix-suffix match
        len = len + 1
        LPS[i] = len
        i = i + 1

    else if len > 0:
        // Fallback to the next longest prefix
        len = LPS[len - 1]

    else:
        // No prefix-suffix match possible at this position
        LPS[i] = 0
        i = i + 1
```

Step 2: Search Pattern in Text

```
i = 0           // Index in text T
j = 0           // Index in pattern P

while i < n:
    if T[i] == P[j]:
        // Characters match, advance both pointers
        i = i + 1
```

```

        j = j + 1

    if j == m:
        // Entire pattern matched
        return i - j

    else if i < n and T[i] != P[j]:
        if j > 0:
            // Reuse previous matches using LPS
            j = LPS[j - 1]
        else:
            // No partial match, move to next character in text
            i = i + 1

return -1    // Pattern not found

```

Complexity

$$\text{Time} = \Theta(n + m), \quad \text{Space} = \Theta(m)$$

Key Invariants (Interview Gold)

- Pointer i (text) never moves backward
- Pointer j (pattern) jumps using LPS to avoid re-comparisons
- Each character is processed at most once

Graph Algorithms: BFS (Breadth-First Search)

What BFS Does (Intuition + When to Use)

Breadth-First Search (BFS) explores a graph in *layers* starting from a source node s : first all nodes at distance 1, then distance 2, etc. It uses a *FIFO queue*.

Key properties (unweighted graphs):

- BFS computes the shortest-path distance (fewest edges) from s to every reachable node.
- The first time a node is discovered is via a shortest path.

Typical interview use-cases:

- Shortest path in an unweighted graph / grid (maze)
- Connected components / reachability
- Level-order traversal (trees)
- Bipartite checking (with coloring)

Complexity (adjacency list):

$$\text{Time} = \Theta(V + E), \quad \text{Space} = \Theta(V)$$

Reason: each vertex is enqueued/dequeued at most once, and each edge is examined at most twice (undirected) or once (directed).

BFS Pseudocode (With Comments)

Assume the graph is given as adjacency list Adj , where $\text{Adj}[u]$ is the list of neighbors of u .

```
BFS(Adj, s):
    // Inputs:
    //   Adj: adjacency list for vertices 0..V-1
    //   s: source/start vertex
    // Output:
    //   dist[v]: shortest distance (#edges) from s to v, or INF if unreachable
    //   parent[v]: predecessor of v on a shortest path (for path reconstruction)

    V = number of vertices in Adj
    dist[0..V-1] = INF
    parent[0..V-1] = NIL

    Q = empty queue

    // Initialize source
    dist[s] = 0
    enqueue(Q, s)

    while Q is not empty:
        u = dequeue(Q)

        // Explore all edges out of u
        for each v in Adj[u]:
            if dist[v] == INF:
                // v not visited yet: first discovery gives shortest path
```

```

    dist[v] = dist[u] + 1
    parent[v] = u
    enqueue(Q, v)

```

```

return dist, parent

```

Path Reconstruction (Optional)

To recover the shortest path from s to a target t (if reachable), follow `parent` pointers backward: $t, \text{parent}[t], \text{parent}[\text{parent}[t]]$ until s , then reverse.

Common Pitfalls

- Marking visited *when dequeued* instead of *when enqueued* can cause multiple enqueues.
- Using adjacency matrix yields $\Theta(V^2)$ time (scan entire row per vertex).
- BFS gives shortest paths only when all edges have equal weight (or weight 1).

Graph Algorithms: DFS (Depth-First Search)

What DFS Does (Intuition + When to Use)

Depth-First Search (DFS) explores a graph by going *as deep as possible* along each branch before backtracking. Conceptually, it follows one path until it cannot continue, then retreats to the most recent branching point.

DFS can be implemented using:

- Recursion (implicit call stack), or
- An explicit stack (iterative DFS)

Typical interview use-cases:

- Reachability and connected components
- Cycle detection (directed and undirected graphs)
- Topological sorting (DAGs)
- Strongly connected components (Kosaraju / Tarjan)
- Backtracking-style graph problems

Important distinction vs BFS: DFS does *not* guarantee shortest paths, but it is excellent for exploring structure and dependencies.

Complexity (adjacency list):

$$\text{Time} = \Theta(V + E), \quad \text{Space} = \Theta(V)$$

Space comes from the recursion stack (or explicit stack) and visited array.

DFS Pseudocode (Recursive, With Comments)

Assume the graph is given as adjacency list Adj.

```
DFS(Adj):
    // Input:
    //   Adj: adjacency list for vertices 0..V-1
    // Output:
    //   visited[v]: whether v has been visited

    V = number of vertices in Adj
    visited[0..V-1] = false

    for v = 0 to V-1:
        if not visited[v]:
            DFS-Visit(v)

DFS-Visit(u):
    // Mark node as visited when first discovered
    visited[u] = true

    // Explore all outgoing edges from u
    for each v in Adj[u]:
        if not visited[v]:
            DFS-Visit(v)
```

DFS with Entry / Exit Times (Very Common Extension)

Useful for cycle detection, topological sort, and SCC algorithms.

```
time = 0

DFS-Visit(u):
    visited[u] = true
    time = time + 1
    tin[u] = time           // entry time

    for each v in Adj[u]:
        if not visited[v]:
            parent[v] = u
            DFS-Visit(v)

    time = time + 1
    tout[u] = time         // exit time
```

Key Properties (Interview Gold)

- Each vertex is visited exactly once
- DFS naturally produces a recursion tree (or forest)
- Back edges indicate cycles in directed graphs

Common Pitfalls

- Stack overflow for deep graphs (mention iterative DFS if needed)

- Forgetting to run DFS from all unvisited nodes (misses components)
- Confusing DFS order with shortest paths (DFS does not minimize distance)

Graph Algorithms: Topological Sort

What Topological Sort Does (Intuition + When to Use)

A *topological order* of a directed graph is a linear ordering of vertices such that for every directed edge $u \rightarrow v$, vertex u appears *before* v in the ordering.

When it applies: Topological sorting is defined for *DAGs* (Directed Acyclic Graphs). If the graph contains a directed cycle, no topological ordering exists.

Typical interview use-cases:

- Dependency resolution (tasks, packages, build systems)
- Scheduling with prerequisites (courses, workflows)
- Detecting cycles in directed graphs (“impossible ordering”)

Complexity (adjacency list):

$$\text{Time} = \Theta(V + E), \quad \text{Space} = \Theta(V)$$

Topological Sort via Kahn’s Algorithm (BFS on In-Degrees)

Idea:

- Compute in-degree for each vertex.
- Start with all vertices with in-degree 0 (no prerequisites).
- Repeatedly remove a zero-in-degree vertex, append it to the order, and decrement in-degrees of its outgoing neighbors.
- If you cannot process all vertices, the graph has a cycle.

TopoSort_Kahn(Adj):

```
// Input:
// Adj[u] = list of outgoing neighbors v for edges u -> v
// Output:
// A topological ordering of all vertices if exists; otherwise "cycle"

V = number of vertices in Adj
indeg[0..V-1] = 0

// 1) Compute indegrees
for u = 0 to V-1:
    for each v in Adj[u]:
        indeg[v] = indeg[v] + 1

// 2) Initialize queue with all sources (indegree 0)
Q = empty queue
for u = 0 to V-1:
    if indeg[u] == 0:
        enqueue(Q, u)

order = empty list
```

```

// 3) Process sources, removing edges as we go
while Q not empty:
    u = dequeue(Q)
    order.append(u)

    // "Remove" u -> v edges by decrementing indegree[v]
    for each v in Adj[u]:
        indeg[v] = indeg[v] - 1
        if indeg[v] == 0:
            enqueue(Q, v)

// 4) If not all vertices were produced, there is a directed cycle
if length(order) != V:
    return "cycle"
else:
    return order

```

Topological Sort via DFS (Postorder)

Idea: Run DFS; when a node finishes (all outgoing edges explored), push it to a list. Reversing that list gives a topological order. A recursion-stack marker can detect cycles.

```

TopoSort_DFS(Adj):
    V = number of vertices in Adj
    visited[0..V-1] = false
    inStack[0..V-1] = false    // recursion stack marker for cycle detection
    order = empty list

    for u = 0 to V-1:
        if not visited[u]:
            if not DFS(u):
                return "cycle"

    reverse(order)
    return order

DFS(u):
    visited[u] = true
    inStack[u] = true

    for each v in Adj[u]:
        if not visited[v]:
            if not DFS(v):
                return false
        else if inStack[v]:
            // Found a back-edge u -> v, which implies a directed cycle
            return false

    inStack[u] = false

    // Postorder append: u must come before nodes that depend on it
    order.append(u)
    return true

```

Which Version to Use in Interviews?

- Use **Kahn's algorithm** when the problem sounds like scheduling / prerequisites.
- Use **DFS** when you already need DFS for something else (e.g., cycle detection), or when you want an ordering from finish times.

Likely Coding Problems (L6 Applied Scientist)

1. Lowest Common Ancestor (Binary Tree)

Problem. Given a binary tree and two nodes p and q , find their *lowest common ancestor* (LCA), defined as the lowest node in the tree that has both p and q as descendants.

Key idea. Use post-order recursion:

- If the current node is null, p , or q , return it.
- Recurse into left and right subtrees.
- If both sides return non-null, the current node is the LCA.
- Otherwise, propagate the non-null result upward.

```
LCA(root, p, q):
    if root == null or root == p or root == q:
        return root

    left = LCA(root.left, p, q)
    right = LCA(root.right, p, q)

    if left != null and right != null:
        return root
    else:
        return left if left != null else right
```

2. Binary Tree Right Side View

Problem. Given a binary tree, return the values of the nodes visible when the tree is viewed from the right side (one value per level, top to bottom).

Key idea. Perform a breadth-first traversal (level order) and record the *last node* encountered at each level.

```
RightSideView(root):
    if root == null:
        return []

    Q = queue([root])
    result = []

    while Q not empty:
        level_size = size(Q)

        for i = 0 to level_size - 1:
            node = pop_left(Q)

            if i == level_size - 1:
                result.append(node.val)

            if node.left != null: push(Q, node.left)
            if node.right != null: push(Q, node.right)

    return result
```

3. Clone Graph

Problem. Given a node in an undirected graph, return a deep copy of the entire connected component reachable from that node.

Key idea. Traverse the graph using BFS (or DFS) while maintaining a map from original nodes to their cloned counterparts. Create each node once, then reuse it via the map.

```
CloneGraph(start):
    if start == null:
        return null

    map = {}                                // original -> clone
    map[start] = new Node(start.val)
    Q = queue([start])

    while Q not empty:
        u = pop_left(Q)

        for v in u.neighbors:
            if v not in map:
                map[v] = new Node(v.val)
                push(Q, v)

            map[u].neighbors.append(map[v])

    return map[start]
```

Coding Problems (LeetCode)

1. Data Processing and Numerical Optimization (Applied Focus)

Sparse Vector Multiplication

Multiply two sparse vectors efficiently by operating only on non-zero entries.

LeetCode: Dot Product of Two Sparse Vectors - done

Large-Scale Aggregation

Given a very large dataset (e.g., tens of millions of records) with an ID column and a quantity column, aggregate quantities per ID efficiently using streaming or hash-based accumulation - done.

LeetCode (pattern reference): Subarray Sum Equals K

Ranking / Scoring Algorithm

Rank items (e.g., movies, products) based on computed scores such as ratings, frequencies, or user interactions, typically requiring sorting or top- K selection.

LeetCode: Top K Frequent Elements

Unfair Coin Simulation

Given access to a biased random generator with unknown probability, design an algorithm that produces fair binary outcomes - done (von Neuman extractor).

LeetCode (related randomness transformation): Implement Rand10() Using Rand7()

Data Structure Design for ML Systems

Design efficient data structures for real-world ML workloads, such as handling high-cardinality categorical features or implementing caching policies - done (heap + doubly linked list).

LeetCode: LRU Cache

2. Graph Algorithms and Tree Traversal

Course Schedule (Cycle Detection / Topological Sort)

Detect whether all courses can be completed given prerequisite constraints, and optionally return a valid ordering using topological sort.

LeetCode: Course Schedule (I), Course Schedule II

Connected Components / Number of Islands

Identify connected components in a graph or grid using DFS or BFS traversal.

LeetCode: Number of Islands

Path Finding (Shortest Path in Grid or Graph)

Find the shortest path between two points in an unweighted grid or graph using BFS.

LeetCode: Shortest Path in Binary Matrix

Lowest Common Ancestor (Binary Tree)

Find the lowest common ancestor of two nodes in a binary tree using recursion or DFS.

LeetCode: Lowest Common Ancestor of a Binary Tree

Tree Traversal and Serialization

Perform various tree traversals such as level-order, zig-zag order, and convert trees to and from serialized representations.

LeetCode:

- Binary Tree Level Order Traversal
- Binary Tree Zigzag Level Order Traversal
- Serialize and Deserialize Binary Tree

3. Strings and Array Manipulation (LeetCode Easy–Medium)

Two Sum / Three Sum Variants

Find pairs or triplets in an array that satisfy a given condition (e.g., sum, product, or target value), typically using hashing or two-pointer techniques.

LeetCode: Two Sum, 3Sum

Unique Element in Sorted Array

Given a sorted array where every element appears exactly twice except one, find the single unique element efficiently.

LeetCode: Single Element in a Sorted Array

String Manipulation and Parsing

Solve common string-processing tasks such as checking balanced parentheses, detecting palindromic permutations, or parsing structured strings.

LeetCode: Valid Parentheses, Palindrome Permutation

K Closest Points to Origin

Given a set of points in a plane, return the K points closest to the origin, typically using sorting or a heap.

LeetCode: K Closest Points to Origin

Subarray Sum Problems

Identify subarrays whose elements satisfy a given sum constraint using prefix sums and hashing.

LeetCode: Subarray Sum Equals K

4. Machine Learning Implementation (Numerical Methods)

Optimization Algorithms (Gradient Descent / Loss Functions)

Implement basic optimization routines such as gradient descent and common loss functions (e.g., mean squared error, cross-entropy) from scratch, focusing on correct numerical updates and stability.

LeetCode (pattern reference): Sqrt(x)

K-Nearest Neighbors / K-Means Clustering

Implement distance-based algorithms such as k -nearest neighbors or k -means clustering, including distance computation and iterative updates.

LeetCode (related patterns): K Closest Points to Origin, Find K Closest Elements - done

Regularization in Loss Functions

Extend a base loss function by adding L1 or L2 regularization terms, and compute gradients accordingly to prevent overfitting and improve stability.

LeetCode (pattern reference): Maximum Average Subarray I

2 Leetcode

LRU cache

Design a data structure that implements a *Least Recently Used (LRU) cache*.

Implement the class `LRUCache` with the following operations:

- `LRUCache(int capacity)`: Initialize the cache with a positive capacity.
- `int get(int key)`: Return the value associated with `key` if it exists in the cache; otherwise, return `-1`.
- `void put(int key, int value)`: If `key` already exists, update its value. Otherwise, insert the key-value pair into the cache. If this insertion causes the number of stored keys to exceed the cache capacity, evict the *least recently used* key.

Both `get` and `put` operations must run in average-case

$$O(1)$$

time.

Course Schedule

There are $n = \text{numCourses}$ courses labeled from 0 to $n - 1$. You are given a list of prerequisite pairs

$$\text{prerequisites}[i] = [a_i, b_i],$$

which indicates that course b_i must be completed *before* course a_i .

For example, the pair $[0, 1]$ means that course 1 is a prerequisite for course 0.

Return `true` if it is possible to complete all courses by satisfying all prerequisites. Otherwise, return `false`.

Course Schedule II

There are $n = \text{numCourses}$ courses labeled from 0 to $n - 1$. You are given a list of prerequisite pairs

$$\text{prerequisites}[i] = [a_i, b_i],$$

which indicates that course b_i must be completed *before* course a_i .

For example, the pair $[0, 1]$ means that course 1 is a prerequisite for course 0.

Return a valid ordering of courses such that all prerequisites are satisfied. If multiple valid orderings exist, return any one of them. If it is impossible to complete all courses (i.e., the prerequisite graph contains a cycle), return an empty array.

200. Number of Islands

Given an $m \times n$ binary grid `grid`, where

$$\text{grid}[i][j] \in \{0, 1\},$$

a value of 1 represents land and a value of 0 represents water.

An *island* is a connected component of land cells (1's) such that:

- land cells are connected horizontally or vertically (not diagonally), and
- the island is completely surrounded by water.

You may assume that all four boundaries of the grid are surrounded by water.

Return the number of distinct islands in the grid.

1091. Shortest Path in Binary Matrix

Given an $n \times n$ binary matrix `grid`, where

$$\text{grid}[i][j] \in \{0, 1\},$$

return the length of the *shortest clear path* from the top-left cell $(0, 0)$ to the bottom-right cell $(n - 1, n - 1)$. If no such path exists, return -1 .

A *clear path* is defined as a path satisfying all of the following conditions:

- All visited cells contain the value 0.
- Consecutive cells in the path are *8-directionally connected*, meaning they share either a side or a corner.

The *length* of a clear path is the total number of cells visited along the path, including both the start and end cells.

236. Lowest Common Ancestor of a Binary Tree

Given a binary tree T , find the *lowest common ancestor* (LCA) of two given nodes p and q in the tree.

According to the definition of LCA:

The lowest common ancestor of two nodes p and q is the lowest node in the tree that has both p and q as descendants, where a node is allowed to be a descendant of itself.

Example 1.

- Input: `root = [3,5,1,6,2,0,8,null,null,7,4]`, $p = 5$, $q = 1$
- Output: 3
- Explanation: The lowest common ancestor of nodes 5 and 1 is node 3.

Example 2.

- Input: `root = [3,5,1,6,2,0,8,null,null,7,4]`, $p = 5$, $q = 4$
- Output: 5
- Explanation: The lowest common ancestor of nodes 5 and 4 is node 5, since a node may be a descendant of itself.

Example 3.

- Input: `root = [1,2]`, $p = 1$, $q = 2$
- Output: 1

102. Binary Tree Level Order Traversal

Given the root of a binary tree, return the *level-order traversal* of the tree's nodes' values, i.e., traversal from left to right, level by level.

Example 1.

- Input: `root = [3,9,20,null,null,15,7]`
- Output: `[[3],[9,20],[15,7]]`

Example 2.

- Input: `root = [1]`
- Output: `[[1]]`

Example 3.

- Input: `root = []`
- Output: `[]`

15. 3Sum

Given an integer array `nums`, return all distinct triplets

$$[\text{nums}[i], \text{nums}[j], \text{nums}[k]]$$

such that

$$i \neq j, \quad i \neq k, \quad j \neq k,$$

and

$$\text{nums}[i] + \text{nums}[j] + \text{nums}[k] = 0.$$

The solution set must not contain duplicate triplets.

Example 1.

- Input: `nums = [-1, 0, 1, 2, -1, -4]`
- Output: `[-1, -1, 2], [-1, 0, 1]`
- Explanation:

$$- (-1) + 0 + 1 = 0$$

$$- 0 + 1 + (-1) = 0$$

$$- (-1) + 2 + (-1) = 0$$

The distinct triplets are `[-1, 0, 1]` and `[-1, -1, 2]`. The order of the output and the order of the triplets does not matter.

Example 2.

- Input: `nums = [0, 1, 1]`
- Output: `[]`
- Explanation: No triplet sums to zero.

Example 3.

- Input: `nums = [0, 0, 0]`
- Output: `[[0, 0, 0]]`
- Explanation: The only possible triplet sums to zero.

20. Valid Parentheses

Given a string `s` consisting only of the characters

`(, ), {, }, [, ]`,

determine whether the input string is *valid*.

An input string is considered valid if all of the following conditions hold:

- Every opening bracket is closed by the same type of bracket.
- Brackets are closed in the correct order.
- Every closing bracket has a corresponding opening bracket of the same type.

Example 1.

- Input: `s = "()"`
- Output: `true`

Example 2.

- Input: `s = "() [] {}"`
- Output: `true`

Example 3.

- Input: `s = "("`
- Output: `false`

Example 4.

- Input: `s = "([])"`
- Output: `true`

Example 5.

- Input: `s = "([)]"`
- Output: `false`